

BankSentinel — Complete Presentation Guide

The Opening Statement (Memorise This)

"In 2016, attackers stole \$81 million from Bangladesh Bank through SWIFT. They stayed inside the network for weeks. Nobody noticed. Nepal runs the same infrastructure — Pumori core banking, the same SWIFT gateways, the same ATM switch networks. Today I am showing you the system that would have stopped them in 38 seconds — not by recognising the attack, but by knowing it was not normal."

What Type of System Is BankSentinel?

BankSentinel is a **Multi-Agent AI-Driven Intrusion Detection System (MAIDS)** — specifically designed for financial infrastructure.

It is not a firewall. It is not a signature scanner. It is not a SIEM dashboard. It is a **threat correlation brain** that sits above all of those and does what none of them can do alone: it fuses signals from five independent detection perspectives into a single, explainable, high-confidence threat decision — and then acts on it automatically in under 500 milliseconds.

The system falls into three academic categories simultaneously:

- **Anomaly-based IDS** — detects deviation from learned normal, not known attacks
 - **Multi-Agent System (MAS)** — five specialised agents collaborate through a shared pipeline
 - **Explainable AI Security System** — every alert carries a human-readable SHAP explanation
-

The Four Problems It Solves

This is your narrative backbone. Every judge question maps back to one of these four.

Problem C1 — Zero-Day Attacks

Signature-based systems like Snort and Suricata maintain libraries of known attack patterns. When a new attack type appears — a zero-day — no signature exists. The system is blind. This is how the Bangladesh Bank heist worked. The tools were all running. None fired.

BankSentinel's answer: The Behavior Agent trains exclusively on 30 days of normal user activity. It has never seen an attack. It does not need to. If a sequence deviates from what normal looks like, reconstruction error spikes and the agent fires. A zero-day attack is still anomalous even if no rule exists for it.

Problem C2 — Legitimate High-Volume False Positives

Every bank runs predictable high-volume operations — ATM reconciliation at 1 AM, RTGS settlement during business hours, month-end batch processing. These superficially resemble DoS attacks and data exfiltration. A global anomaly model would generate a 22.7% false positive rate during ATM reconciliation windows alone. Analysts receiving 48,200 alerts per day stop trusting the system entirely.

BankSentinel's answer: The Flow Agent trains six separate Isolation Forest models — one per Nepal banking time context. The model that runs at 1 AM Nepal time was trained on legitimate ATM reconciliation traffic. It knows that high volume at that hour is normal. False positive rate drops from 22.7% to 2.1%.

Problem C3 — Alert Fatigue

Banking SOC's receive tens of thousands of alerts daily. Studies show analysts become desensitized after roughly 200 unactionable alerts. They start dismissing warnings reflexively. This is not laziness — it is a known psychological response to information overload. The Bangladesh Bank SOC received alarms. They were ignored.

BankSentinel's answer: The Correlation Agent applies four suppression mechanisms before a single notification reaches an analyst. Raw alert volume drops 87%. The 48,200 alerts per day become 6,350 — of which 497 are genuinely high priority and only 7.1% are false positives. An analyst can actually respond to 497 alerts. Nobody can respond to 48,200.

Problem C4 — Encrypted TLS 1.3 Traffic

Modern C2 frameworks — Cobalt Strike, Metasploit, custom APT tooling — communicate exclusively over TLS 1.3. Payload inspection is mathematically infeasible. Traditional deep packet inspection is dead. The attacker simply encrypts the channel and traditional IDS becomes blind to the most dangerous traffic class.

BankSentinel's answer: The Packet Agent operates on three independent layers, none of which require decryption. Layer 1 fingerprints the TLS handshake itself using JA3 hashing — the cipher suite choices, extension order, and curve preferences form a signature unique to each TLS client implementation. Layer 2 extends this to the server side — even when attackers randomise their client fingerprint, the server response pattern (JA3S) reveals Cobalt Strike profiles. Layer 3 catches true zero-day TLS by analysing timing — C2 malware calls home on a schedule, producing abnormally low inter-arrival time variance. Human HTTPS browsing is unpredictable. Malware is not.

The Five Agents — Deep Explanation

Agent 1 — Packet Agent

What it does: First line of detection. Analyses every network connection at the metadata level without touching payload.

How it works: Three independent detection layers. Layer 1 queries the live abuse.ch JA3 threat database — 14,200+ known-malicious TLS fingerprints refreshed every 30 minutes. A Cobalt Strike C2 connection from the SWIFT subnet matches a known fingerprint and fires immediately. Layer 2 catches attackers who have read the JA3 papers and changed their client

fingerprint — the server-side JA3S fingerprint is harder to randomise and reveals the same C2 framework. Layer 3 catches everything else by analysing beacon timing using a Random Forest trained on CTU-13 real botnet C2 traffic.

Why it matters: This agent is the only component of the entire system that catches threats inside encrypted sessions. Without it, TLS 1.3 traffic is a complete blind spot. With it, a Cobalt Strike connection from a SWIFT subnet workstation is caught within 2 seconds of the first handshake.

What makes it unique: The live feed integration. When you run the demo, the JA3 database was updated minutes ago — not months ago. No other team in this room has live threat intelligence.

Key metric: 91.7% detection rate on CTU-13 TLS scenarios. Baseline flow statistics alone: 73.2%. JA3 alone: 81.4%. The three-layer approach adds 10 percentage points over the best single-method baseline.

Agent 2 — Flow Agent

What it does: Analyses network flow statistics to detect anomalous traffic patterns — volume spikes, unusual port usage, new external connections, lateral movement indicators.

How it works: Six Isolation Forest models trained on institution-specific NetFlow data, one per Nepal banking time context. The key insight is that what is anomalous at 3 PM on a Tuesday is completely normal at 1 AM during ATM reconciliation. Routing each flow to the contextually appropriate model eliminates the endemic false positive problem. Each model was trained with a different contamination parameter reflecting the expected anomaly base rate for that time window.

Why it matters: Without this agent, the system has no view of network-level volume patterns. It cannot detect port scans, internal reconnaissance, lateral movement between subnets, or data exfiltration by volume. It also provides the global model comparison that proves the C2 false positive reduction claim — the number judges will put in their notes.

What makes it unique: Calendar awareness. The system knows it is ATM reconciliation time in Nepal. It knows the difference between legitimate month-end batch processing and an exfiltration attempt that happens to coincide with it. No other IDS in the market is calibrated for Nepal banking operational patterns.

Key metric: ATM reconciliation window false positive rate — 22.7% with a global model, 2.1% with context-aware routing. This is your most visually striking proof point.

Agent 3 — Behavior Agent

What it does: Monitors user and entity behavior — login patterns, privilege usage, database query rates, lateral authentication sequences — and flags deviations from each account's learned normal profile.

How it works: A Bidirectional LSTM autoencoder trained exclusively on normal behavioral sequences. The model learns to reconstruct normal behavior with low error. When an anomalous sequence arrives — a service account executing 400 database queries in 120 seconds, an employee logging in from a new IP at 2 AM, a standard user account suddenly appearing in privileged groups — the reconstruction error spikes. The detection threshold is set at the 95th percentile of normal training errors. No attack labels. No rules. Pure deviation detection.

Why it matters: Insider threats and credential abuse are the hardest attack category to detect because the attacker is using legitimate accounts. Signature-based systems have essentially zero detection capability here — there is no signature for an employee accessing one more database than usual. The Behavior Agent detects the anomaly in the behavioral pattern itself.

What makes it unique: It is entirely unsupervised with respect to attacks. You can add a new attack type tomorrow — a technique invented next year — and the Behavior Agent will still detect it if it deviates from normal. The model never needs to be retrained to catch new attack types. This is the defining characteristic that separates it from rule-based UEBA systems like Splunk UBA.

Key metric: 94.9% overall detection rate across five zero-day attack scenarios, 1.3% false positive rate. No attack signatures used at any stage of training.

Agent 4 — Correlation Agent

What it does: The brain of the system. Receives alerts from all three detection agents, fuses them using a Bayesian Belief Network, computes a Composite Risk Score, applies four alert suppression mechanisms, and produces the final threat assessment for the SOC analyst.

How it works: A Bayesian Belief Network models the joint probability that a true threat campaign is occurring given alerts from the Packet, Flow, and Behavior agents. The prior probability is calibrated to the empirical banking threat base rate — 3.2 incidents per 10,000 events. When all three agents fire simultaneously, the posterior probability of a genuine threat campaign approaches 95%. The Composite Risk Score combines this Bayesian probability with per-agent confidence scores using weights derived from empirical performance. Four suppression mechanisms then operate on the alert stream before any notification reaches an analyst — deduplication (collapsing repeated signatures within 5 minutes), causal chaining (merging alerts that share a source IP into one incident ticket), confidence gating (holding back low-confidence single-agent alerts from immediate notification), and context-aware filtering (auto-closing alerts that match known legitimate operational patterns with full audit logging).

Why it matters: Without this agent, you have three independent detection systems each generating their own stream of alerts. An analyst receiving 1,000 alerts from the Packet Agent, 2,000 from the Flow Agent, and 500 from the Behavior Agent simultaneously has no way to know they are all describing the same attack campaign. The Correlation Agent answers the question: *are these three signals describing one coordinated attack, or three unrelated events?* That question is unanswerable without multi-source fusion.

What makes it unique: The alert fatigue reduction is the single most operationally significant number in the entire system. 87% alert volume reduction while maintaining 100% campaign-level detection rate. The mathematical proof that you can suppress more than four out

of five alerts without missing any real attack campaign is the result that will resonate most with banking security professionals in the room.

Key metric: Raw alerts per day from 48,200 to 6,350. Campaign-level detection rate unchanged at 100%.

Agent 5 — Response Agent

What it does: Executes automated containment actions when the Composite Risk Score exceeds 0.85, and maintains a tamper-evident forensic audit trail for every action.

How it works: Network quarantine via ACL updates completes in under 500 milliseconds. Account suspension calls the Active Directory API. Suspicious external IPs are pushed to perimeter firewall blocklists within 2 seconds. A forensic PCAP snapshot is triggered on the identified flows and exported in STIX 2.1 format. An NRB-format compliance report is auto-generated as a PDF. The SOC dashboard receives a prioritised alert with the full SHAP explanation. All actions are logged to an append-only audit chain using SHA-256 hash chaining — Equation 5 from the paper — making every record tamper-evident and satisfying PCI-DSS 10.3 requirements.

Why it matters: Detection without response is just observation. The Bangladesh Bank heist lasted three weeks because detection was slow and response was manual. BankSentinel reduces mean analyst response time from 14.2 minutes to 3.8 minutes — and for the highest-confidence threats, automated containment fires before any human has read the alert.

What makes it unique: The compliance integration. Every action automatically produces an NRB-format incident report citing the specific NRB Cybersecurity Guidelines sections, SWIFT CSP controls, and PCI-DSS requirements satisfied. No analyst manually fills out a compliance form. The audit chain satisfies PCI-DSS 10.3 on its own — judges who understand banking regulation will immediately recognise this as production-ready, not prototype-ready.

Key metric: Network quarantine in under 500ms. Mean analyst response time reduction from 14.2 to 3.8 minutes. Full audit trail satisfying PCI-DSS 10.3 and NRB Cybersecurity Guidelines Section 4.5.

How BankSentinel Differs from Everything Else

Dimension			Snort / Suricata	Splunk UBA	Darktrac e	BankSentinel
Zero-day detection			✗ Signature only	Partial	Partial	✓ Full (C1)
TLS 1.3 visibility			✗ Payload blind	✗	Limited	✓ 3-layer JA3
Calendar-aware FPR			✗	✗	✗	✓ 6 contexts
Alert fatigue			✗ Raw volume	Limited	Limited	✓ 87% reduction
Nepal regulatory alignment			✗	✗	✗	✓ NRB/SWIFT/PCI
Auto NRB compliance report			✗	✗	✗	✓
Pumori/SWIFT context			✗	✗	✗	✓ Built-in

Explainability (SHAP)	✗	Limited	✗	✓ Per-alert
Cost	High	Very high	Very high	Open source stack

The most important differentiator is not technical. It is contextual. Darktrace, Splunk, and CrowdStrike are built for Western financial infrastructure. They do not know what ATM reconciliation traffic looks like at 1 AM Nepal time. They do not know the NRB Cybersecurity Guidelines. They do not know that Pumori uses specific database ports. They have never been calibrated to the Nepal banking threat landscape.

BankSentinel was built from the ground up for this environment. That is not a marketing claim. It is in every line of the code.

Business Pitch

The Problem in Business Terms

Every Nepalese commercial bank currently operates with one or more of these gaps:

- A signature-based IDS that cannot detect novel attacks
- A SIEM that generates so many alerts that analysts stop reading them
- Zero visibility into encrypted SWIFT and C2 communications
- Manual compliance reporting that costs hours per incident
- No integration between network-level anomalies and user-behavior anomalies

The cost of getting this wrong is not hypothetical. The Bangladesh Bank heist caused \$81 million in direct losses. The reputational damage was incalculable. Nepal's banking sector manages over NPR 6 trillion in assets. A similar attack on NIC Asia or Everest Bank would be catastrophic not just for the institution but for public confidence in the entire financial system.

The Solution in Business Terms

BankSentinel is not a product. It is infrastructure. It is what you build once and operate continuously. It replaces the fragmented patchwork of signature scanners, manual alert reviews, and spreadsheet-based compliance tracking with a single unified platform that:

- **Detects threats that no existing tool in Nepal can detect** — zero-day attacks, encrypted C2 channels, insider behavioral anomalies
- **Reduces the cost of running a SOC** — 87% fewer alerts means analysts spend their time on real threats, not false alarms
- **Produces compliance documentation automatically** — NRB incident reports, PCI-DSS audit logs, SWIFT CSP evidence packages
- **Responds in milliseconds** — not minutes, not hours — automatically isolating compromised endpoints before lateral movement can begin

Pricing Reality Check

A commercial deployment of Darktrace in a mid-sized bank costs USD 200,000 to 500,000 per year. Splunk Enterprise Security adds another 100,000 to 300,000. Neither is calibrated for Nepal. Neither speaks NRB. Neither knows what Pumori looks like from the inside.

BankSentinel runs on open-source components — Python, scikit-learn, PyTorch, FastAPI, PostgreSQL, Elasticsearch. The infrastructure cost for a single-bank deployment is hardware and engineering time. The per-bank marginal cost of the federated consortium model — where anonymised threat signatures are shared across NRB member banks — approaches zero.

The Vision

The federated IOC sharing stub in the codebase is not an afterthought. It is the long-term value proposition. When NIC Asia detects a new attack pattern, that anonymised signature reaches Everest Bank and NMB Bank within seconds — without sharing any customer data, without regulatory complications, without anyone reading anyone else's transactions.

Nepal's banking sector has 27 commercial banks. Each one faces the same threat landscape. Each one is currently defending individually with inadequate tools. BankSentinel is the first step toward a coordinated, AI-native, Nepal-specific collective defence.

That is the pitch. Not a product. National infrastructure.

Answering Judge Questions

"How is this different from just running Snort?"

Snort fires on signatures — patterns of known attacks. BankSentinel fires on deviation from learned normal. Snort would have missed the Bangladesh Bank attack completely because there was no signature for it. BankSentinel's Behavior Agent would have fired when svc_corebanking executed 400 database queries in 120 seconds — because that deviates from its 30-day learned baseline, regardless of whether any rule exists.

"Your XGBoost got 100% accuracy — isn't that suspicious?"

Yes, and I noted that in my review. The Friday Bot dataset is highly separable — the Bot traffic has distinct statistical signatures that Random Forest and XGBoost can learn nearly perfectly on that specific train-test split. In production, the Packet Agent does not use XGBoost as primary detection. It uses the CTU-13 Random Forest for beacon timing analysis and the live JA3 feed for fingerprint matching. The XGBoost result demonstrates feature importance understanding, not production performance.

"What happens when an attacker knows about JA3 and rotates fingerprints?"

Layer 3 catches them. When an attacker randomises their TLS client fingerprint to evade Layer 1, Layer 3 still catches the beacon timing pattern — the malware calls home every 60 seconds regardless of what TLS fingerprint it uses. You cannot hide the fact that you are calling home on a schedule by changing your cipher suite order. That is why we built three independent layers.

"How does this handle privacy? You are analysing employee behavior."

The Behavior Agent works on anonymised behavioral sequences — event types, time patterns, query rates. It does not read email content, document contents, or communication text. IP addresses in the audit log are pseudonymised using HMAC-SHA256 with rotating keys. Only authorised SOC personnel and compliance officers can access individual behavioral profiles.

This is compliant with GDPR Article 25 (privacy by design) and is explicitly aligned with the NRB data protection requirements.

"Is this ready for production?"

The inference pipeline, the alert structure, the audit chain, and the compliance reporting are production-ready code. The training notebooks are Google Colab-based for the hackathon. A production deployment would replace Colab with a scheduled retraining pipeline running on the bank's own GPU infrastructure or a private cloud. The Kubernetes Helm chart is stubbed in the architecture. This is a production-ready prototype, not an academic exercise.

"Why should a bank trust a system trained on synthetic behavioral data?"

The Behavior Agent's architecture is deliberately designed to require very little real attack data — it is trained only on normal sequences. In a real deployment, normal sequences come from 30 days of actual Windows Event Log data from the bank's own Active Directory. The model learns that bank's specific normal patterns — not synthetic ones. The synthetic generator in the codebase is a stand-in for the hackathon. Plug in real Event Logs and the model immediately becomes specific to that institution.

The Demo Script — What to Say While It Runs

As the APT scenario fires on screen:

"Watch the top left panel — that is the network topology of a Nepalese commercial bank. Green nodes are healthy. I am about to simulate the exact attack sequence from Section III of the paper — the same type of coordinated attack that hit Bangladesh Bank.

At time zero — a workstation in the SWIFT messaging subnet initiates an outbound TLS connection to an IP in Eastern Europe. The Packet Agent fires immediately — Layer 1 identifies the JA3 fingerprint as a known Cobalt Strike C2 profile from the live threat feed. That node just turned yellow.

Two seconds later — the Flow Agent detects that SWIFT subnet traffic is running at 4.2 times the off-hours baseline. It routes this to the off-hours context model — not the global model — because it is 3 AM Nepal time. The context model knows this deviation is not ATM reconciliation. Yellow is becoming orange.

Four seconds later — the Behavior Agent detects that svc_corebanking has executed 400 SELECT queries against the Pumori database in the last 120 seconds. No rule exists for this pattern. The BiLSTM has never seen this sequence in 30 days of normal operation. Reconstruction error spikes. The node turns red.

Eight seconds from the first packet — the Correlation Agent fuses all three signals through the Bayesian Belief Network. Composite Risk Score: 0.94. Critical. 847 related alerts have been collapsed into this single incident ticket. The Response Agent has already isolated 10.22.14.45 from the network.

Total elapsed time: 38 seconds. From first packet to containment.

The Bangladesh Bank attackers had three weeks. This bank would have had 38 seconds of exposure."

Final Summary Card

When a judge asks you to summarise in one sentence:

"BankSentinel is Nepal's first AI-native intrusion detection system — five specialised agents that together solve the four core problems no existing tool addresses: zero-day attacks with no known signature, false positives during legitimate banking operations, alert fatigue from unmanageable alert volumes, and threats hidden inside encrypted TLS traffic — all integrated with NRB regulatory compliance reporting and designed specifically for Nepal's SWIFT, Pumori, and ATM infrastructure."